Analisis Operativo del Proceso de Desarrollo

Intrale Platform — Monorepo Kotlin (Ktor + Compose Multiplatform)

Fecha: 18 de Marzo de 2026 | Sprint Activo: SPR-041 | Sprints Completados: 40+

CONFIDENCIAL v1.0

Resumen Ejecutivo

Este informe analiza la operacion completa del proceso de desarrollo de Intrale Platform, un sistema que combina **agentes IA autonomos** (Claude Code) con orquestacion automatizada via hooks, scripts PowerShell y notificaciones Telegram. Se identifican **7 fortalezas clave**, **9 debilidades** y **5 puntos de atencion urgente**, junto con recomendaciones concretas para optimizar el consumo de tokens, reducir la complejidad operativa y mejorar la confiabilidad del pipeline.

Indice

1. Panorama del Sistema Actual
2. Metricas Operativas Clave
3. Fortalezas del Proceso
4. Debilidades Identificadas
5. Puntos de Atencion Urgente
6. Analisis de Eficiencia en Tokens
7. Propuesta: Migracion de Agentes a Scripts
8. Lecciones Aprendidas Historicas
9. Plan de Mejora Priorizado
10. Conclusion

1. Panorama del Sistema Actual

1.1 Arquitectura de Automatizacion

El sistema opera en **6 capas de automatizacion**:

Capa	Componentes	Funcion
T0 - Manual	`Start-Agente.ps1`	Lanzamiento de agentes por sprint

T1 - Skills	27 skills en `.claude/skills/`	Capacidades especializadas (/backend-dev, /qa, etc.)
T2 - Hooks Sync	PreToolUse gates, PostToolUse loggers	Validacion en tiempo real + telemetria
T3 - Hooks Async	agent-concurrency-check, scrum-monitor-bg	Gestion de cola + salud del sprint
T4 - Monitors	agent-watcher.js, ci-monitor-bg.js	Health checks en background
T5 - Dashboard	dashboard-server.js @ :3100	Visibilidad via Puppeteer + Telegram

1.2 Composicion del Ecosistema

Componente	Cantidad	Peso Aprox.
Hooks JavaScript (.js)	58 archivos	~450 KB total
Skills (.claude/skills/)	27 directorios (25 activos, 2 congelados)	~200 KB SKILL.md total
Scripts PowerShell	5 principales + auxiliares	~80 KB
Scripts Node.js (scripts/)	15+ planificacion/reporte	~180 KB
Agentes rules (agents/)	21 archivos markdown	~50 KB
Archivos de estado (.json)	15+ archivos de estado mutable	Variable
TOTAL infraestructura IA	**~140 archivos**	**~960 KB**

2. Metricas Operativas Clave

40+
Sprints completados

58
Hooks activos

27
Skills configurados

3
Max agentes simultaneos

5
Historias por sprint

~377K
Tokens estimados/24h

2.1 Distribucion de Modelos por Rol ACTUALIZADO

Modelo	Skills Asignados	Uso Tipico	Costo Relativo

Opus	backend-dev, android-dev, web-dev, qa, tester, review + sesion principal	Implementacion, generacion de tests, code review, QA E2E, orquestacion	Alto
Sonnet	guru, planner, po, ux, security, hotfix	Investigacion, planificacion, analisis de producto	Medio
Haiku	branch, doc, delivery, monitor, auth, builder, ops, historia, refinar, priorizar, cleanup, scrum, cost, perf	Tareas operativas, deterministas, bajo razonamiento	Bajo

Correccion Aplicada: Reasignacion de Modelos (2026-03-18)

Se detectó que **Opus no se usaba en ningun agente de desarrollo**, desperdiciando el modelo mas capaz en solo la sesion principal. Se corrigieron 6 skills:

Skill	Antes	Ahora	Justificacion
/backend-dev	Sonnet	**Opus**	Implementacion backend completa requiere razonamiento profundo
/android-dev	Sonnet	**Opus**	Compose UI + arquitectura MVVM compleja
/web-dev	Sonnet	**Opus**	Kotlin/Wasm con particularidades de compilacion
/qa	Sonnet	**Opus**	Generacion de casos de prueba E2E + analisis de resultados
/tester	**Haiku**	**Opus**	**Caso mas critico:** generaba tests con el modelo mas basico
/review	Sonnet	**Opus**	Code review profundo detecta bugs sutiles

Impacto esperado: Mejor calidad de codigo generado, tests mas exhaustivos, menos iteraciones de correccion post-QA, reviews que detecten mas problemas antes del merge.

2.2 Health Check — Estado Actual

Componente	Estado	Checks Consecutivos OK
Commander	PASS	25
Approvers	PASS	25
Telegram	PASS	25
Settings	PASS	25

Hooks	PASS	47
Worktrees	PASS	25

3. Fortalezas del Proceso

F1. Automatizacion Madura y Multi-capa

Con 58 hooks activos, el sistema cubre validacion pre-ejecucion (branch-guard, delivery-gate), telemetria post-ejecucion (activity-logger, agent-metrics), y auto-reparacion (scrum-auto-corrections, ensure-permissions). Esto reduce drasticamente errores humanos.

Evidencia: 47 checks consecutivos sin fallas en hooks; inconsistencias detectadas y reparadas automaticamente en SPR-025, SPR-028, SPR-033.

F2. Fuente Unica de Verdad (roadmap.json)

Desde la correccion del issue #1621 (SPR-040), `roadmap.json` es la unica fuente de verdad para sprints. `sprint-plan.json` es un cache auto-generado. Esto resolvio el problema historico de desincronizacion entre multiples archivos de estado.

Impacto: Elimina una clase entera de bugs de consistencia que causaron sprints fantasma y carry-overs perdidos.

F3. Pipeline de QA Riguroso

Gate obligatorio `QA E2E → Tester → PO` con labels verificables (`qa:passed` / `qa:skipped`) antes de merge. El hook `delivery-gate.js` bloquea automaticamente merges sin QA. Videos de evidencia enviados a Telegram como parte del cierre.

Impacto: Cero regresiones en produccion desde implementacion del gate.

F4. Sizing Abstracto (Simple/Medio/Grande)

Abandonar estimaciones temporales (dias/semanas) por complejidad abstracta es una decision excelente para un equipo operado por agentes IA, donde el tiempo real depende de slots, rate limits y dependencias externas.

F5. Sistema de Permisos Auto-healing

Defensa en 4 capas: baseline (50 allow + 7 deny), hook auto-heal cada 1h, Guardian cada 5 min, y auto-learn on-the-fly. Esto garantiza que agentes nuevos arrancan con permisos correctos sin intervencion manual.

F6. Transparencia Total via Telegram

Notificaciones automaticas para: completitud de tareas, errores bloqueantes, aprobaciones de permisos, CI status, y heartbeats periodicos con screenshots del dashboard. El operador tiene visibilidad completa desde el celular.

F7. Worktree Isolation por Agente

Cada agente trabaja en su propio worktree Git, eliminando conflictos de archivos entre agentes concurrentes. La leccion de junctions NTFS (2026-02-27) fue bien aprendida: ahora se usan copias de `.claude/` en vez de symlinks.

4. Debilidades Identificadas

D1. Complejidad Excesiva del Ecosistema de Hooks (CRITICA)

Problema: 58 archivos JavaScript de hooks (~450 KB) representan un sistema operativo paralelo que:

- Consume contexto de Claude en cada sesion (los hooks se cargan como parte del entorno)
- Tiene interdependencias no documentadas (activity-logger lanza dashboard-server, que usa telegram-client, que depende de telegram-config)
- Genera overhead en cada tool use: `permission-gate.js` evalua CADA accion con timeout de 1830s
- Dificulta el debugging: cuando algo falla, hay 6+ capas donde buscar

Dato: `agent-concurrency-check.js` tiene 55 KB — mas grande que muchos microservicios del backend.

D2. Consumo de Tokens en Tareas Operativas

Problema: Skills operativos que no requieren razonamiento avanzado (monitor, scrum, ops, cleanup, branch, delivery) consumen tokens de Claude para tareas que podrian ser scripts deterministas.

Ejemplo: `/monitor` recopila estado de 12 paneles leyendo archivos JSON y formateando output — una tarea puramente mecanica. `/scrum` valida consistencia entre roadmap.json y GitHub issues — logica determinista.

Estimacion de desperdicio: ~30-40% de los tokens de una sesion de orquestacion se gastan en lectura/formateo de estado, no en razonamiento.

D3. Agent Watcher Deshabilitado (Bug #1551)

Problema: El `agent-watcher.js` fue deshabilitado porque evaluaba PRs a los 3 segundos, marcando agentes como "failed" antes de que trabajaran. Esto deja un hueco en la supervision automatica: si un agente muere silenciosamente, no hay deteccion automatica.

Impacto: Dependencia en supervision manual para detectar agentes muertos.

D4. Rate Limits con 3 Agentes Simultaneos

Problema: 3 agentes Claude corriendo en paralelo frecuentemente alcanzan rate limits de la API, especialmente cuando usan modelo Sonnet para implementacion. La leccion de SPR-029/030 documenta esto, pero la mitigacion actual (escalonamiento manual de 2-3 min) es fragil.

Impacto: Agentes se ralentizan o fallan por throttling, reduciendo throughput efectivo.

D5. Falsos Positivos en QA Operativo

Problema documentado (2026-03-09): Agentes cerraban issues de QA verificando que el codigo existia (grep/ls) sin ejecutarlo realmente. Patron: buscar archivo → encontrar → concluir "implementado" → cerrar issue.

Mitigacion parcial: Regla de "ejecutar funcionalidad, no solo verificar codigo" documentada, pero depende de que el agente la siga — no hay enforcement automatico.

D6. Proliferacion de Archivos de Estado

Al menos **15 archivos JSON/JSONL mutables** en `.claude/hooks/` : agent-metrics, agent-registry, scrum-monitor-state, health-check-state, health-check-components, ops-learnings, scrum-health-history, telegram-messages, pending-questions, activity-logger-last, worktree-guard-last-alert, auto-review-state, agent-progress-state, launcher-last-check, sprint-pids. Mas `roadmap.json` y `sprint-plan.json` en scripts/.

Riesgo: Corrupcion, race conditions entre hooks concurrentes, y dificultad para diagnosticar inconsistencias.

D7. Skills SKILL.md como Contexto Pesado

Los SKILL.md son prompts extensos (el de monitor tiene 440 lineas, planner 100+, backend-dev 100+). Cuando se invoca un skill, todo el SKILL.md se carga en contexto. Para skills que se invocan frecuentemente, esto multiplica el consumo de tokens.

D8. Dependencia de Puppeteer para Screenshots

El dashboard envia heartbeats via Puppeteer (headless Chrome → screenshot PNG → Telegram). Este es un proceso pesado que consume RAM significativa y tiene fallos documentados (isPngValid, consecutiveSkipCount). Una solucion mas ligera seria viable.

D9. Documentacion Fragmentada de la Infraestructura

Las reglas operativas estan distribuidas entre: 42 archivos de memoria, CLAUDE.md, 21 archivos en agents/, SKILL.md de cada skill, y comentarios en los hooks. No hay un documento unico que explique "como funciona todo junto".

5. Puntos de Atencion Urgente

U1. Overhead de Hooks en Cada Tool Use [ALTA]

Situacion: Cada llamada a herramienta pasa por `permission-gate.js` (PreToolUse) y `activity-logger.js` + `post-tool-orchestrator.js` (PostToolUse). Con agentes que hacen 80-150 tool calls por sesion, esto suma miles de ejecuciones de hooks por sprint.

Riesgo: Latencia acumulada + potencial de errores en cascada. Si un hook falla, puede bloquear al agente.

Recomendacion: Auditar latencia real de cada hook. Considerar batching o sampling para activity-logger (logear 1 de cada N acciones en vez de todas).

U2. CI Failures Silenciosos [ALTA]

Situacion: El agente #677 tuvo CI failure pero el PR #1652 fue mergeado exitosamente. ops-learnings registra el fallo pero no hubo bloqueo.

Riesgo: Merges con CI rojo pasan al main branch. El delivery-gate verifica labels QA pero no status de CI.

Recomendacion: Agregar check de CI status en delivery-gate.js ademas de labels QA.

U3. Worktrees Huerfanos Recurrentes [MEDIA]

Situacion: 11 worktrees muertos detectados el 2026-03-04 (19 ocurrencias en ops-learnings). Aunque fueron limpiados, el issue subyacente sigue OPEN — los agentes que mueren no siempre limpian su worktree.

Recomendacion: Agregar cleanup en el hook Stop para el agente que termina, no solo en health-check periodico.

U4. 18+ Ramas Agent/* Legacy en Remoto [MEDIA]

Situacion: Ramas de sprints anteriores siguen en el repositorio remoto. Esto ensucia el namespace y puede confundir herramientas.

Recomendacion: Script de limpieza periodica de ramas mergeadas. Puede ser un cron o parte del cierre de sprint.

U5. Sin Metricas de Costo Real por Sprint [MEDIA]

Situacion: Aunque existe `cost-report.js` y se estiman ~377K tokens/24h, no hay tracking preciso de costo USD por sprint ni por historia. Esto dificulta decisiones de optimizacion.

Recomendacion: Integrar metricas de costo de la API de Anthropic al cierre de sprint.

6. Analisis de Eficiencia en Tokens

6.1 Distribucion Estimada del Consumo

Categoria	% Estimado	Tokens/Sprint	Optimizable?
Implementacion de codigo (backend-dev, android-dev, web-dev)	35%	~500K	Parcialmente (mejores prompts)
QA E2E y testing	15%	~215K	Si — scripts Maestro son deterministas
Orquestacion y planificacion (scrum, planner, monitor)	20%	~285K	**Si — alto potencial**
Operaciones (ops, cleanup, branch, delivery)	15%	~215K	**Si — mayoría determinista**
Carga de contexto (SKILL.md, CLAUDE.md, memory)	10%	~140K	Si — reducir SKILL.md
Hooks overhead (permission-gate, activity-logger)	5%	~70K	Si — sampling/batching

6.2 Top Oportunidades de Ahorro

#	Oportunidad	Ahorro Estimado	Esfuerzo
1	Migrar /monitor a script Node.js puro	~50K tokens/sprint	Medio
2	Migrar /scrum validacion a script Node.js	~40K tokens/sprint	Medio
3	Migrar /ops health-check a script Node.js	~30K tokens/sprint	Simple

4	Migrar /cleanup a script bash/node	~20K tokens/sprint	Simple
5	Migrar /branch a script git puro	~15K tokens/sprint	Simple
6	Reducir SKILL.md de skills frecuentes	~30K tokens/sprint	Simple
7	Delivery como script con gh CLI	~25K tokens/sprint	Medio
	TOTAL POTENCIAL	**~210K tokens/sprint**	

Nota: 210K tokens/sprint representa aproximadamente un 15% de reduccion en consumo total.

7. Propuesta: Migracion de Agentes a Scripts

7.1 Criterio de Migracion

Un skill es candidato a migracion total o parcial si cumple:

1. **Determinista:** La logica no requiere razonamiento — solo lectura de estado, transformacion y escritura
2. **Repetitivo:** Se ejecuta multiples veces por sprint con la misma logica
3. **Sin creatividad:** No necesita generar codigo, analizar problemas o tomar decisiones ambiguas

7.2 Clasificacion de Skills

Skill	Clasificacion	Accion Recomendada
/monitor	**100% Determinista**	**Migrar completamente** a script Node.js. Ya lee archivos JSON y formatea output.
/scrum	**90% Determinista**	**Migrar validacion y sync** a script. Mantener agente solo para decisiones de reparacion complejas.
/ops	**95% Determinista**	**Migrar completamente.** health-check.js ya hace el trabajo real.
/cleanup	**100% Determinista**	**Migrar completamente.** Borrar worktrees/logs/procesos es puramente mecanico.
/branch	**100% Determinista**	**Migrar completamente.** git worktree add + git checkout es un script de 20 lineas.
/delivery	**80% Determinista**	**Migrar parcialmente.** Commit+push+PR es mecanico, pero el mensaje de commit requiere analisis de cambios.
/builder	**85% Determinista**	**Migrar parcialmente.** gradlew build es mecanico, diagnosticar fallas requiere razonamiento.
/tester	**70% Determinista**	**Migrar ejecucion** de tests a script. Mantener agente para analisis de fallas y generacion de tests.
/cost	**100% Determinista**	**Migrar completamente.** Calculo de metricas puras.
/backend-dev	10% Determinista	Mantener como agente. Requiere razonamiento completo.
/android-dev	10% Determinista	Mantener como agente.

/qa	20% Determinista	Mantener como agente. La ejecucion de Maestro ya es un script, pero el analisis de resultados requiere IA.
/review	15% Determinista	Mantener como agente. Code review necesita razonamiento.
/guru	5% Determinista	Mantener como agente. Investigacion pura.
/po, /ux	5% Determinista	Mantener como agente. Analisis de producto/UX.

7.3 Arquitectura Propuesta Post-Migracion

Capa	Antes	Despues
Operaciones	8 skills IA (monitor, scrum, ops, cleanup, branch, delivery, builder, cost)	8 scripts Node.js/bash invocables sin Claude
Implementacion	5 skills IA (backend-dev, android-dev, web-dev, qa, review)	Sin cambio — requieren razonamiento
Producto	4 skills IA (po, ux, guru, doc)	Sin cambio — requieren creatividad
Orquestacion	Sesion Opus principal	Script orquestador + Opus solo para decisiones

8. Lecciones Aprendidas Historicas

De las 42 memorias operativas acumuladas, las lecciones mas impactantes fueron:

#	Leccion	Origen	Impacto	Estado
1	**Fuente unica de verdad para sprints** — roadmap.json como single source, sprint-plan.json como cache	Bug critico: sprint entero ejecutado sin que roadmap lo supiera	CRITICO	RESUELTO (#1621)
2	**Junctions NTFS destruyen .claude/** — usar copias, no symlinks	rm -rf en worktree borro .claude/ real del repo principal	CRITICO	RESUELTO
3	**Agent watcher con grace period** — no evaluar antes de 15 min	Agentes marcados "failed" a los 3 segundos de arrancar	CRITICO	PARCIAL (watcher deshabilitado)
4	**No usar Agent tool para implementar** — usar terminales independientes	Subprocesos internos invisibles, no auditables	ALTO	RESUELTO

5	**QA debe ejecutar, no solo verificar existencia**	Agentes cerraban issues confirmando que archivos existian sin probarlos	CRITICO	PARCIAL (regla, sin enforcement)
6	**Re-leer archivos despues de escribir** — hooks pueden revertir	Hooks sobreescribian cambios silenciosamente	ALTO	RESUELTO
7	**OpenAI key perdida al cambiar de branch**	Config no commiteada + branch switch = perdida de credenciales	ALTO	RESUELTO
8	**Cola no debe mover agentes recien promovidos a incomplete**	Bug cascada: cola se vaciaba sola	ALTO	RESUELTO
9	**Rate limits con 3 agentes** — considerar escalonamiento	SPR-029/030: agentes throttled	ALTO	PARCIAL (manual)
10	**PowerShell stream-json con errores de sintaxis** — crash silencioso	PR #1550 revertido	ALTO	RESUELTO (revert a Start-Transcript)

8.1 Patron de Problemas Recurrentes

Analizando las lecciones, emergen 3 patrones:

1. **Complejidad genera bugs de consistencia:** Multiples fuentes de verdad (roadmap vs sprint-plan), hooks que interfieren entre si, estados distribuidos en 15+ archivos JSON.
2. **Automatizacion sin enforcement real:** Reglas documentadas (ej: "QA debe ejecutar") que dependen de que el agente las siga, sin validacion mecanica.
3. **Windows/MSYS2 como fuente de friccion:** Junctions NTFS, paths con espacios, heredocs, PowerShell vs bash — cada uno genero al menos un bug critico.

9. Plan de Mejora Priorizado

Prioridad	Mejora	Esfuerzo	Impacto	Sprint Sugerido
P0	**Migrar /monitor a script Node.js** — eliminar consumo de tokens para lectura de estado puro	Medio	~50K tokens/sprint ahorro	SPR-042
P0	**Agregar CI status check en delivery-gate.js** — prevenir merges con CI rojo	Simple	Elimina regresiones por CI ignorado	SPR-042
P1	**Migrar /branch, /cleanup, /cost a scripts** — tareas 100% deterministas	Simple	~85K tokens/sprint ahorro	SPR-042
P1	**Rehabilitar agent-watcher con grace period 15min** — deteccion automatica de agentes muertos	Simple	Elimina supervision manual	SPR-042
P1	**Consolidar archivos de estado** — reducir de 15+ JSONs a 3-4 archivos maestros	Grande	Reduce race conditions y simplifica debugging	SPR-043
P2	**Migrar /scrum validacion y /ops a scripts**	Medio	~70K tokens/sprint ahorro	SPR-043
P2	**Implementar rate-limit-aware scheduling** — escalonamiento automatico de agentes basado en consumo real de API	Medio	Elimina throttling, mejora throughput	SPR-043
P2	**Limpieza periodica de ramas remotas** — script post-sprint que elimina agent/* mergeadas	Simple	Namespace limpio	SPR-042
P2	**Reducir SKILL.md de skills frecuentes** — comprimir instrucciones a lo esencial	Simple	~30K tokens/sprint ahorro en contexto	SPR-043
P3	**Dashboard ligero sin Puppeteer** — reemplazar screenshots PNG por reportes texto/HTML enviados directamente	Grande	Reduce RAM + elimina errores PNG	SPR-044
P3	**Documento unico de arquitectura de infra IA** — como funciona todo junto	Medio	Reduce onboarding friction	SPR-044

9.1 Impacto Acumulado del Plan

~235K	8→3	15→4

Tokens/sprint ahorro potencial	Skills operativos que usan IA	Archivos de estado JSON

10. Conclusion

Veredicto General

El proceso de desarrollo de Intrale tiene una **base solida**: automatizacion madura, QA riguroso, fuente unica de verdad, y transparencia total. Sin embargo, ha alcanzado un punto de **complejidad donde la infraestructura de IA consume recursos significativos en tareas que no requieren inteligencia**.

La principal oportunidad de mejora es **separar lo determinista de lo inteligente**: migrar tareas mecanicas (monitor, scrum, ops, cleanup, branch) a scripts puros, reservando tokens de Claude exclusivamente para lo que realmente necesita razonamiento (implementacion, review, QA analisis, producto).

El plan propuesto de 11 mejoras distribuidas en 3 sprints lograria un ahorro estimado de **~235K tokens por sprint** (15-17% del total) mientras mejora la confiabilidad al eliminar dependencias innecesarias de IA para operaciones rutinarias.

Generado automaticamente por Claude Opus 4.6 — Intrale Platform
18 de Marzo de 2026